**PROJECT REPORT OF CO-OP Project at Industry (Module-2)**

**ON**

# SMART BILLING MANAGEMENT SYSTEM

**submitted in partial fulfilment of the requirements for the award of degree of**

**BACHELOR OF ENGINEERING**

**In**

**COMPUTER SCIENCE AND ENGINEERING**


**Submitted by:**

**Tanishq Garg (2210990899)**

**Preksha Mahajan (2210992083)**

**Avantika (2210990197)**

**Shiven Narang (2210990835)**

**Supervised By:**

**Dr. Rajat Takkar**

**Assistant Professor**

**CUIET, Punjab**




**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY**
**CHITKARA UNIVERSITY, PUNJAB, INDIA**

# CONTENTS

# DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"Smart Billing Management System"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Rajat Takkar**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura
Date: 29 April 2026

**Tanishq Garg (2210990899)**
**Preksha Mahajan (2210992083)**
**Avantika (2210990197)**
**Shiven Narang (2210990)**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

**Dr. Rajat Takkar**
Assistant Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Rajat Takkar, Assistant Professor at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab for continuous guidance, support, and valuable feedback throughout the development of this project.

I am also thankful to the faculty members of the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for their encouragement and academic support during the CO-OP program.

Finally, I thank my teammates and friends for their collaboration, testing support, and constructive suggestions.

# ABSTRACT

Smart Billing Management System (SBMS) is a MERN-stack web application designed to support small businesses and shops in managing inventory, creating bills, tracking sales and purchase analytics, and collecting payments. The system provides role-based access for Admin and User accounts. Admins can manage stock, categories/subcategories, and view business analytics, while users can create invoices by adding items to a cart, completing payment, and generating a PDF invoice.

The backend is implemented using Node.js and Express.js with MongoDB as the database via Mongoose ODM. It includes authentication using JWT tokens (also stored as HTTP-only cookies), inventory CRUD operations with optional image upload, cart management, invoice generation (PDF + email delivery), payment integration using Razorpay, and testimonial APIs. The frontend is a Create React App (React 18) application using TailwindCSS for UI, Redux + redux-persist for login state, Chart.js for analytics visualizations, and jsPDF for invoice PDF generation.

This report describes the methodology, tools, implementation details, major outcomes, and future scope of SBMS.

# 1. Introduction

## 1.1. Background and Motivation

Billing and inventory tracking are core operational activities in retail and small businesses. Manual processes (paper invoices, spreadsheets) are error-prone, time-consuming, and make it difficult to analyze sales, purchases, or low-stock situations. SBMS addresses these challenges by providing a single platform for inventory, billing, and analytics.

## 1.2. Problem Statement

To design and develop a web-based system that:

- Maintains inventory items with categories, subcategories, pricing, quantities, and optional images.
- Enables users to create bills by selecting items, computing totals, and generating invoices.
- Supports payment collection and verification.
- Provides dashboards to view stock summaries and business metrics.

## 1.3. Objectives

- Implement secure authentication and role-based access control (admin/user).
- Provide inventory CRUD and low-stock detection.
- Provide cart-based billing workflow.
- Integrate payment gateway (Razorpay) and store payment records.
- Generate professional invoice PDFs and optionally email them.
- Provide analytics such as month-wise purchase totals and sales totals.

## 1.4. Scope

The scope of SBMS includes:

- Web UI for landing pages, login/signup, and a protected dashboard.
- Backend APIs for auth, inventory, invoices, cart, categories/subcategories, payments, and testimonials.

- Database persistence for users, inventory, cart, invoices, payments, categories, and subcategories.

# 2. Methodology

## 2.1. Development Approach

An iterative approach was followed:

1) Requirements identification (inventory + billing + payment + analytics).

2) Module-wise implementation (Auth → Inventory → Cart/Billing → Payments → Analytics).

3) Integration between frontend and backend APIs.

4) Testing using manual API testing and UI verification.

## 2.2 System Design Method

SBMS follows a client–server architecture:

- Client: React-based SPA for UI and workflows.
- Server: Express REST API for business logic.
- Database: MongoDB for persistence.

## 2.3 Data Flow Summary

User actions (login, inventory operations, cart updates, payment) trigger REST requests to backend endpoints. The backend validates authentication (where required), performs database operations, and returns JSON responses. For billing, the frontend generates a PDF invoice and the backend can also generate and email an invoice PDF.

# 3. Tools and Technologies

## 3.1. Frontend

- React (Create React App)
- React Router (routing)
- Redux + redux-thunk (state management)
- redux-persist (persist login session)
- TailwindCSS (styling)
- Chart.js + react-chartjs-2 (charts)
- jsPDF + jspdf-autotable (PDF invoice generation)
- Axios / Fetch (API calls)

## 3.2. Backend

- Node.js + Express.js (server)
- MongoDB + Mongoose (database/ODM)
- JWT (authentication)
- cookie-parser, cors, body-parser (middleware)
- Multer (file upload for inventory images)
- Nodemailer (emails for password reset / invoice)
- html-pdf (invoice PDF generation in backend)
- Razorpay SDK + crypto (payment checkout and verification)

## 3.3. Deployment/Hosting (as used in codebase)

- Frontend deployed on Vercel (URLs present in CORS allow-list)
- Backend referenced as deployed on Render in frontend API calls

# 4. Implementation

## 4.1. Project Structure

SBMS consists of two main projects:

A) Backend:

- index.js: Express app, middleware configuration, and route mounting
- config/database.js: MongoDB connection
- routers/: REST routes for auth, inventory, invoice, cart, categories/subcategories, payments, testimonials
- controllers/: Business logic
- models/: Mongoose schemas (User, Inventory, Cart, Invoice, Payment, Category, Subcategory, etc.)
- middlewares/: JWT auth and multer upload

B) Frontend:

- src/App.js: Application routes (public + dashboard)
- src/Layout.js: Protected layout (requires authenticated user)
- src/Dashboard/*: Dashboard modules (inventory list, low stock, billing, analytics, settings)
- src/components/*: Login/Signup/Forgot/Reset password, inventory form, payment success
- src/Redux/*: Auth store and actions

## 4.2. Authentication and Authorization

Backend auth features:

- Signup (POST /api/v1/auth/signup) with roles: admin or user.
- Login (POST /api/v1/auth/login) issues JWT and sets token cookie.
- Middleware auth validates JWT from cookie/body/Authorization header.
- Role checks: isAdmin, isUser.

Frontend auth features:

- Login uses Redux action and stores JWT token in localStorage.

- Protected dashboard: Layout.js redirects to /login if the user is not authenticated.

## 4.3. Inventory Management

Inventory API endpoints include:

- Create item with image upload (POST /api/v1/inventory/createinventory)
- Update item (PUT /api/v1/inventory/inventory/:id)
- Delete item (DELETE /api/v1/inventory/inventory/:id)
- List all inventory (GET /api/v1/inventory/getallinventory)
- Low stock items (GET /api/v1/inventory/getlowinventory) based on quantity threshold
- Search (GET /api/v1/inventory/search?query=...)
- Analytics endpoints: category-wise stock, purchase value, monthly purchase totals

Inventory schema stores name, category, subcategory, quantity, unit, purchase price, selling price, and file path.

## 4.4. Category and Subcategory Management

Category and subcategory modules support:

- CRUD for categories (/api/v1/categoryy/categories)
- CRUD for subcategories (/api/v1/categoryy/subcategories)

These are used in frontend forms to provide structured inventory classification.

## 4.5. Cart and Billing Workflow

Billing is cart-driven:

- Users search inventory items and add them to cart (POST /api/v1/cart/add).
- Cart can be fetched (GET /api/v1/cart), items removed (DELETE /api/v1/cart/remove), and cleared (DELETE /api/v1/cart/clear).
- On cart clear, backend also updates inventory quantities by subtracting sold quantities.

## 4.6. Invoice Generation

Two invoice mechanisms are present:

1) Frontend: Generates a professional invoice PDF using jsPDF and jspdf-autotable.

2) Backend: Can generate an invoice PDF using html-pdf and email it using nodemailer.

## 4.7. Payments (Razorpay)

Payment implementation includes:

- Checkout (POST /api/v1/pay/checkout) creates Razorpay order.

- Verification (POST /api/v1/pay/paymentverification) validates signature using HMAC SHA256 and stores payment records.

- Reporting endpoints: total amount (GET /api/v1/pay/getTotal), month-wise selling (GET /api/v1/pay/allsell), and per-user payments (GET /api/v1/pay/getspe/:id).

## 4.8. Testimonials

Testimonials module provides:

- Fetch testimonials (GET /api/v1/testimonial/)

- Create testimonial (POST /api/v1/testimonial/)

# 5. Major Findings/Outcomes/Output/Results

## 5.1. Functional Outcomes

- Role-based dashboard navigation (admin vs user).
- Inventory CRUD with category/subcategory classification and image support.
- Low-stock view to help plan restocking.
- Cart-based billing, tax and discount computation, and invoice PDF generation.
- Razorpay payment flow with backend verification and payment history.
- Business analytics dashboard with chart visualizations for purchases and sales.

## 5.2. Non-Functional Outcomes

- Modular backend structure (routes/controllers/models) improved maintainability.
- Uses JWT-based authentication and protected routes.
- Cross-origin configuration supports local dev and deployed frontend domains.

# 6. Conclusion and Future Scope

## 6.1. Conclusion

SBMS successfully delivers an end-to-end solution for inventory management and billing with payment integration. The application demonstrates a complete MERN-stack workflow including database persistence, authentication, role-based UI, invoice generation, and analytics.

## 6.2. Future Scope

1. **Mobile Application:** Developing a React Native or Flutter app to allow on-the-go inventory management and billing from smartphones, especially useful for field staff.

2. **Barcode/QR Code Scanning:** Integrating a barcode scanner for faster product lookup and checkout, reducing manual entry errors at the point of sale.

3. **Multi-store/Multi-branch Support:** Extending the system to manage multiple store locations under a single admin account with centralized analytics.

4. **GST-Compliant Invoicing:** Adding GST number support, HSN codes, and GSTIN-based tax breakdowns to make invoices legally compliant for Indian businesses.

5. **Demand Forecasting with ML:** Using historical sales data to predict future stock requirements, helping businesses avoid both overstocking and stockouts.

# REFERENCES


1. React Documentation (Create React App)
2. Express.js Documentation
3. MongoDB and Mongoose Documentation
4. Razorpay API Documentation
5. JSON Web Token (JWT) Documentation
6. Nodemailer Documentation